AURUM: An AI-Powered Multi-Agent System for Intelligent Evaluation of Gold (XAUUSD) Trading Strategies

Faculty Incharge: Dr. Chinmayananda A

Group Members:

- Mohammad Danish Ansari (22BDS039)
- Aashish Saini (22BEC002)
- Ambrish Pandey (22BEC007)

GitHub Repository: https://github.com/danishansari-dev/AURUM

Abstract

When retail traders try to use multiple technical indicators at once, they often get confusing or contradictory signals. To solve this, we built **AURUM**, an AI-powered system that lets you type out Gold (XAUUSD) trading strategies in plain English to see how well they actually work.

AURUM uses a hybrid parser (Regex combined with Claude AI) to break down these sentences and routes the rules to five separate "specialist agents"one each for RSI, EMA, SMA, MACD, and Bollinger Bands. An Orchestrator then reviews their feedback, checks for logical conflicts (like buying when momentum is crashing), and gives the strategy a final score out of 100. At the same time, a high-speed backtesting engine runs the strategy through five years of historical data to calculate real-world metrics like Win Rate and Maximum Drawdown.

Everything is tied together in an interactive Streamlit dashboard. During our testing, AURUM successfully recognized famous, profitable setups like the Golden Cross, correctly penalized flawed strategies, and stopped contradictory trades that standard platforms blindly execute.

Keywords: Multi-Agent System, Technical Analysis, XAUUSD Gold, Natural Language Processing, Backtesting, Strategy Evaluation, Conflict Detection, Streamlit Dashboard.

1. Introduction

1.1 Background and Motivation

The global foreign exchange and commodity markets process trillions of dollars in daily volume, with spot Gold (XAUUSD) consistently ranking among the most actively traded instruments [1]. Technical Analysis (TA), the practice of predicting future price movements from historical price and volume data using mathematical indicators, remains the predominant methodology among retail traders [2]. Indicators such as the Relative Strength Index (RSI) [3], Moving Average Convergence Divergence (MACD), Exponential Moving Average (EMA), Simple Moving Average (SMA), and Bollinger Bands each capture different facets of market behaviour — momentum, trend, and volatility respectively.

However, the application of these indicators in isolation or in poorly-considered combinations introduces two fundamental problems that existing backtesting platforms fail to address adequately:

1. **Lack of Explainability:** Platforms such as TradingView and MetaTrader execute strategy rules mechanically and report aggregate profit-and-loss metrics, but provide no per-indicator reasoning that explains *why* a strategy succeeds or fails. A trader whose RSI-based strategy underperforms cannot determine whether the issue lies in the threshold choice, the historical reliability of the RSI at that threshold level on Gold specifically, or in conflicting momentum dynamics captured by other indicators.
2. **Absence of Conflict Detection:** When a strategy combines multiple indicators, logical contradictions can arise that traditional platforms execute blindly. For instance, an RSI reading below 30 signals an oversold (bullish) condition, while a concurrent MACD bearish crossover signals accelerating downward momentum. Executing trades under such contradictory signals leads to unpredictable outcomes, yet no existing retail platform intercepts or warns about these contradictions at strategy-design time.

1.2 Problem Statement

Retail traders and finance students designing multi-indicator trading strategies for the Gold (XAUUSD) market currently lack an automated system capable of: (a) evaluating each indicator condition against domain-specific rules and historical performance data, (b) detecting logical contradictions between indicator signals, (c) providing per-indicator explainable feedback, and (d) presenting quantitative backtest evidence alongside qualitative improvement suggestions within a unified interface.

1.3 Proposed Solution

To address this gap, this paper introduces **AURUM** - an AI-powered, modular multi-agent framework in which each technical indicator is represented by an autonomous Specialist Agent. The system architecture follows the Multi-Agent System (MAS) paradigm [4], wherein agents operate independently on their respective indicator domains, communicate

results through a typed message protocol, and delegate coordination to a central OrchestratorAgent. The Orchestrator performs weighted score aggregation, inter-agent conflict detection, and verdict generation. The complete pipeline - from natural language input to interactive dashboard output - executes within seconds, enabling rapid iterative strategy refinement.

1.4 Contributions

The principal contributions of this work are:

1. A dual-mode NLP strategy parser combining deterministic regex matching with Anthropic Claude LLM fallback, producing Pydantic-validated condition structures.
2. A polymorphic specialist agent architecture where each agent applies a three-component scoring model: rule-based credit (40 points), historical win-rate estimation (40 points), and divergence detection (20 points).
3. A concurrent orchestration engine using `ThreadPoolExecutor` for parallel agent dispatch with an `AgentMessage` protocol for auditable inter-agent communication.
4. A multi-pattern conflict detection subsystem identifying action-alignment mismatches, RSI-overbought/EMA-uptrend contradictions, and MACD-bearish/RSI-oversold ambiguities.
5. An end-to-end interactive dashboard integrating live Gold price feeds, strategy evaluation, vectorised backtesting, and detailed agent report panels.

1.5 Paper Organisation

Section 2 describes related work and the theoretical background of the indicators employed. Section 3 presents the system architecture and design decisions. Section 4 details the implementation methodology for each subsystem. Section 5 discusses experimental results and testing. Section 6 concludes the paper and outlines future work.

---

2. Related Work and Theoretical Background

2.1 Multi-Agent Systems in Finance

Multi-Agent Systems (MAS) have been extensively studied in computational finance for tasks ranging from market simulation [5] to portfolio optimisation [6]. The MAS paradigm decomposes complex problems into cooperating autonomous agents, each possessing specialised knowledge and operating under bounded rationality. In the context of trading strategy evaluation, a MAS architecture provides natural modularity: each indicator's evaluation logic is self-contained, testable in isolation, and extensible without cross-cutting impacts on other agents.

2.2 Technical Indicators for Gold

Gold exhibits distinctive market microstructure characteristics - it functions simultaneously as a commodity, a currency, and a safe-haven asset - that influence the reliability of different technical indicators:

- **RSI (Relative Strength Index):** Wilder [3] proposed RSI as a momentum oscillator bounded between 0 and 100. Conventional oversold (< 30) and overbought (> 70) thresholds are empirically effective on Gold due to its strong momentum-driven behaviour during geopolitical events.
- **MACD (Moving Average Convergence Divergence):** Appel [7] designed MACD to capture momentum changes through the convergence and divergence of two exponential moving averages (default 12/26/9). MACD crossovers and divergences have demonstrated statistically significant predictive power on daily Gold charts.
- **EMA/SMA (Exponential/Simple Moving Averages):** The Golden Cross (50-period crossing above 200-period moving average) and its inverse, the Death Cross, are widely recognised trend signals. EMAs weight recent prices more heavily than SMAs, providing faster trend responsiveness for Gold's volatile intraday behaviour.
- **Bollinger Bands:** Bollinger [8] constructed bands at two standard deviations around a 20-period moving average, producing a volatility envelope. When Gold's price touches the lower band, a mean-reversion buy signal is generated; the Bollinger Squeeze (narrowing bandwidth) signals impending directional breakouts.

2.3 Backtesting Methodologies

Event-driven backtesting engines (e.g., Backtrader, Zipline) iterate bar-by-bar, realistically simulating order fills but incurring significant computational overhead. Vectorised approaches, exemplified by the `vectorbt` library [9], exploit NumPy array operations to evaluate entire signal arrays simultaneously, achieving orders-of-magnitude performance improvements. Since AURUM targets rapid evaluation iteration on daily data, the vectorised approach was selected for the backtesting subsystem.

2.4 NLP for Financial Strategy Parsing

Rule extraction from natural language in the financial domain has been addressed through template-based methods [10], dependency parsing, and more recently through large language models (LLMs). AURUM employs a pragmatic hybrid: compiled regular expressions handle canonical indicator phrases with zero latency, while an LLM fallback (Anthropic Claude) addresses complex or ambiguous sentence constructions.

3. System Architecture

3.1 Architectural Overview

AURUM follows a layered, modular architecture decomposed into four principal layers:

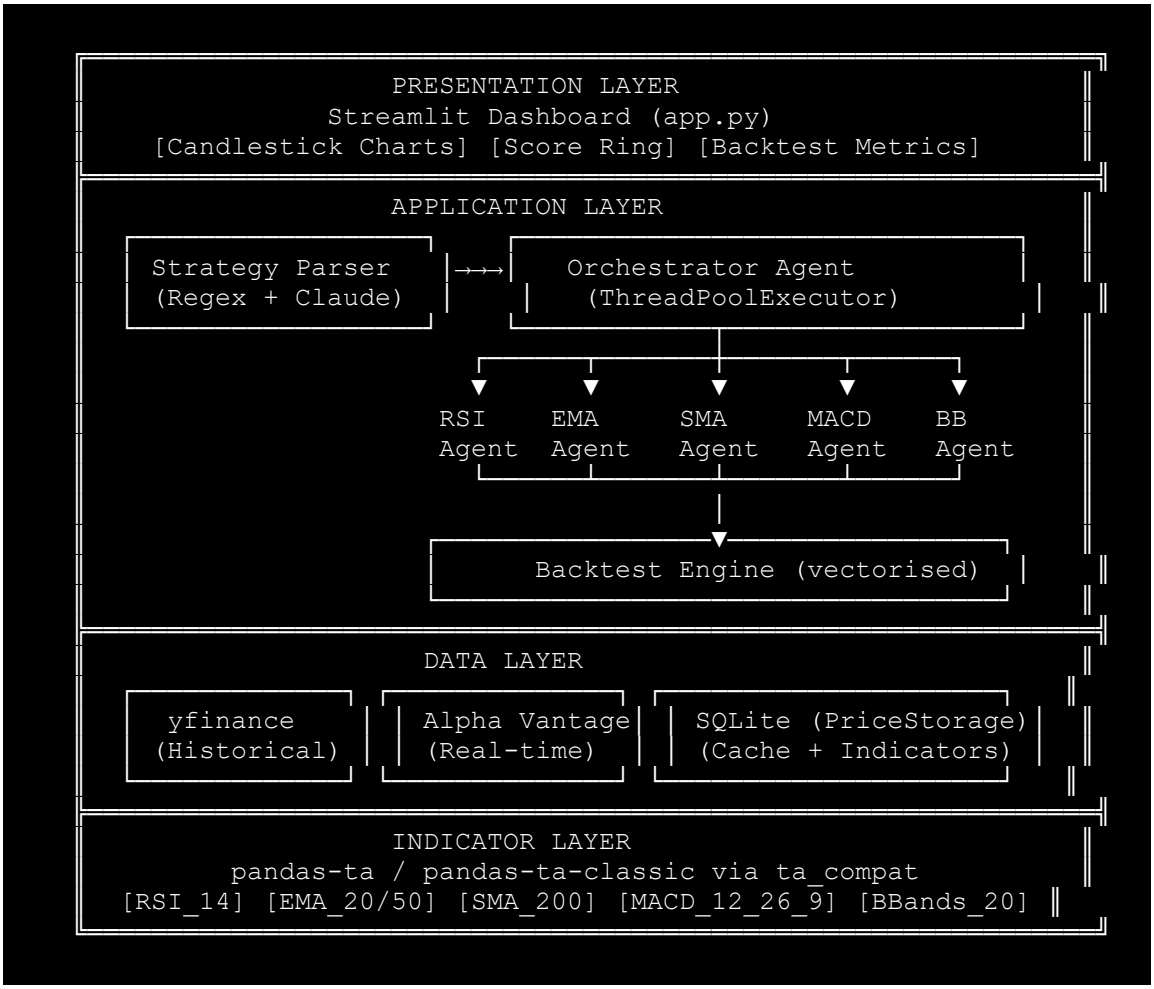

Fig. 1. High-level architecture of the AURUM system.

3.2 Design Principles

The architecture was governed by the following design principles:

Principle	Realisation
Separation of Concerns	Each agent owns one indicator domain and communicates only through the Orchestrator via typed `AgentMessage` envelopes.

Principle	Realisation
Open/Closed Principle	New agents (e.g., Stochastic, Ichimoku) can be added by subclassing `BaseAgent` without modifying existing agents or the Orchestrator's routing logic.
Fail-Safe Degradation	When the Anthropic API key is absent, parsing falls back to regex-only mode. When yfinance's primary symbol `XAUUSD=X` is unavailable, the fetcher retries with `GC=F` (Gold Futures).
Reproducibility	The SQLite `PriceStorage` layer caches OHLCV data and precomputed indicators, ensuring identical evaluation results across runs without network dependency.
Performance	The `ThreadPoolExecutor`-based fan-out evaluates all five agents in parallel, and vectorised backtesting avoids bar-by-bar iteration.

3.3 Inter-Agent Communication Protocol

All communication between Specialist Agents and the OrchestratorAgent is wrapped in a typed `AgentMessage` dataclass:

```
    @dataclass
    class AgentMessage:
        sender: str                                         # "RSIAgent"
        receiver: str                                       #
"OrchestratorAgent"
        msg_type: Literal["SCORE", "CONFLICT_FLAG", "DATA_REQUEST"]
        payload: dict[str, Any]
        timestamp: datetime
```

This protocol provides a complete audit trail of every exchange during an evaluation pass, enabling post-hoc analysis of scoring decisions, conflict triggers, and agent interaction ordering.

3.4 Agent Result Schema

Every Specialist Agent returns a Pydantic-validated `AgentResult` model:

```
    class AgentResult(BaseModel):
        agent_name: str                     # Logical identifier
        score: float                        # 0-100 composite
        win_rate: float                     # [0.0, 1.0] historical
```

```
success rate
        feedback: list[str]                # Narrative observations
        suggestions: list[str]            # Actionable next steps
        action_alignment: Literal["BUY", "SELL", "NEUTRAL"]
```

The `score` field is bounded to `[0, 100]` via a `_safe_score()` clamp. The `win_rate` field is validated through a `field_validator` that coerces raw floats into `[0, 1]`. The `action_alignment` field controls how the Orchestrator's conflict detector classifies each agent's directional stance.

---

4. Implementation and Methodology

4.1 Data Layer

4.1.1 Historical Data Fetching

The `fetch_xauusd_history()` function leverages the `yfinance` library to download multi-year daily OHLCV data for the `XAUUSD=X` symbol. The function enforces timezone-aware UTC indexing, lowercase column normalisation, and validates the presence of all five required columns (open, high, low, close, volume). A runtime exception is raised when zero rows are returned, guiding the user to the alternative `GC=F` symbol.

```
    def fetch_xauusd_history(period="5y", interval="1d") ->
pd.DataFrame:
        ticker = yf.Ticker("XAUUSD=X")
        raw = ticker.history(period=period, interval=interval, ...)
        # ... normalise, validate, tz-localise to UTC
        return frame
```

4.1.2 Real-Time Price Feed

The `fetch_realtime()` function queries the Alpha Vantage `CURRENCY_EXCHANGE_RATE` endpoint for the XAU/USD pair. The API key is resolved from the function parameter or the `ALPHA_VANTAGE_API_KEY` environment variable. Since the free-tier endpoint returns a single quoted rate rather than an intraday OHLC bar, the function synthesises a flat candle (`open = high = low = close = rate`) to maintain schema consistency with the historical data interface.

4.1.3 SQLite Persistence (PriceStorage)

The `PriceStorage` class implements a context-managed SQLite persistence layer with two tables:

- **`price_data`**: Daily OHLCV bars keyed by ISO date string, with `UPSERT` semantics (`ON CONFLICT(date) DO UPDATE`).
- **`indicator_cache`**: Precomputed indicator columns (RSI, EMA20, EMA50, SMA200, MACD line/signal, Bollinger upper/lower) aligned by date, also using `UPSERT` semantics.

This caching layer eliminates redundant yfinance API calls during iterative strategy refinement sessions and ensures the system remains functional under network interruptions.

4.2 Indicator Computation Layer

The `add_all_indicators()` function in `indicators/calculator.py` appends nine indicator columns to an OHLCV DataFrame using the `pandas-ta` library:

Indicator Column	Computation	Parameters
`rsi`	Relative Strength Index	length=14
`ema20`	Exponential Moving Average	length=20
`ema50`	Exponential Moving Average	length=50
`sma200`	Simple Moving Average	length=200
`macd_line`	MACD Line	fast=12, slow=26, signal=9
`macd_signal`	MACD Signal Line	fast=12, slow=26, signal=9
`bb_upper`	Upper Bollinger Band	length=20, std=2
`bb_lower`	Lower Bollinger Band	length=20, std=2

Table 1. Technical indicators computed by the AURUM indicator layer.

A cross-version compatibility shim (`ta_compat.py`) imports `pandas_ta` when available and falls back to `pandas_ta_classic` for Python versions below 3.12, maintaining a uniform functional API across deployment environments.

4.3 NLP Strategy Parser

4.3.1 Dual-Mode Architecture

The `StrategyParser` class implements a two-tier parsing strategy:

Mode 1 — Regex Parser: Twenty-two compiled regular expressions (compiled at class definition time for amortised cost) match canonical indicator phrases across six pattern categories:

- RSI threshold comparisons (`RSI below 30`, `RSI > 70`)
- EMA crossovers (`EMA20 crosses above EMA50`)
- EMA-vs-price comparisons (`price above EMA50`)
- SMA comparisons and crossovers
- MACD bullish/bearish crossover phrases
- Bollinger Bands upper/lower band touches

Each match is coerced into a raw condition dict with standardised keys: `indicator`, `operator`, `value`, `fast`, `slow`, and `action`. Trade direction (`BUY`/`SELL`) is inferred from contextual keywords (`buy`, `long`, `bullish` → BUY; `sell`, `short`, `bearish` → SELL), defaulting to BUY when the text is ambiguous.

Mode 2 — Claude AI Fallback: When the regex parser yields fewer than two conditions (indicating either a simple query or a complex sentence the patterns failed to capture), the system falls back to the Anthropic Claude API (`claude-sonnet-4-20250514`). The LLM receives a constrained system prompt instructing it to return only a JSON array of condition objects conforming to the `StrategyCondition` schema. The raw response is stripped of markdown code fences, the outermost `[...]` block is extracted, and the result is JSON-deserialized.

4.3.2 Pydantic Validation

Every raw condition dict - whether from regex or LLM - passes through the `StrategyCondition` Pydantic model:

```
class StrategyCondition(BaseModel):
    indicator: str       # Normalised to uppercase
    operator: str
    value: float | None  # Threshold for comparisons
    fast: int | None     # Fast period (crossovers)
    slow: int | None     # Slow period (crossovers)
    action: Literal["BUY", "SELL"]
```

A `model_validator` enforces that EMA crossover conditions carry both `fast` and `slow` periods, and that `fast < slow`. Invalid conditions are logged and silently dropped, ensuring one malformed clause does not invalidate the entire strategy.

4.4 Specialist Agents

All five Specialist Agents inherit from the abstract `BaseAgent` class, which mandates two abstract methods: `evaluate_condition()` and `calculate_win_rate()`. Each agent's scoring model follows a uniform three-component decomposition summing to a maximum of 100 points.

4.4.1 RSI Agent

The RSI Agent evaluates momentum conditions through three scoring components:

1. **Rule-Based Credit (40 points):** The latest RSI reading is mapped against domain-derived tier boundaries. For BUY actions: RSI < 30 earns 40 points (classic oversold), RSI < 35 earns 30 points, RSI < 40 earns 15 points, and RSI < 45 earns 5 points. SELL actions follow a symmetric mapping around the overbought threshold at RSI > 70.
2. **Historical Win-Rate Estimation (40 points):** The agent scans all historical bars where the RSI condition was satisfied, then measures the fraction of setups where the `close` price moved at least 0.5% in the signal direction within a 5-bar forward horizon. The win-rate fraction is multiplied by 40 to yield the ML component score.
3. **Divergence Detection (20 points):** The agent inspects the most recent 20 bars for classic RSI-price divergence patterns. For BUY: bullish divergence is detected when price makes a lower low while RSI makes a higher low across two identified swing troughs. For SELL: bearish divergence requires higher price highs with lower RSI highs. Detection uses a dual-pass algorithm — first attempting trough/peak identification via local minima/maxima, then falling back to half-window comparison.

Action Alignment: If the composite score exceeds 55, the agent aligns with the condition's action direction (BUY or SELL); otherwise, it reports NEUTRAL, signalling insufficient conviction to influence the Orchestrator's conflict analysis.

4.4.2 MACD Agent

The MACD Agent evaluates crossover and histogram-based conditions:

1. **Rule-Based Credit (40 points):** Points are awarded based on three independent checks — MACD line vs. signal line position (20 points for alignment with action), histogram polarity

(10 points), and MACD position relative to the zero line (10 points for strong trend confirmation).

2. **Historical Win-Rate (40 points):** The agent builds a boolean mask from histogram polarity (`hist > 0` for BUY, `hist < 0` for SELL) and measures the 5-bar, 0.5% forward success rate identically to the RSI Agent.

3. **MACD-Price Divergence (20 points):** Bullish divergence is detected when the second half of a 20-bar lookback window shows a lower price minimum but a higher MACD minimum. Bearish divergence follows the symmetric pattern for maxima.

Additionally, the agent exposes a standalone `is_bearish_crossover()` method that detects actual crossover events (MACD crossing below signal between the last two bars), used by the Orchestrator's conflict subsystem.

4.4.3 EMA Agent

The EMA Agent (source: `ema_agent.py`, 283 lines) evaluates two condition subtypes:

- **Crossover Conditions:** Compares fast EMA (e.g., EMA20) against slow EMA (e.g., EMA50). The agent recognises canonical combinations - (9,21) for scalping, (20,50) for short-medium crossovers, and (50,200) for the Golden/Death Cross - and assigns base scores reflecting their empirical reliability on Gold.
- **Price-vs-EMA Conditions:** Evaluates whether the close price is above or below a specified EMA, with supplementary slope analysis measuring the EMA's directional slope over a 5-bar lookback to grade trend strength (strong, moderate, weak, or flat).

4.4.4 SMA Agent

The SMA Agent (source: `sma_agent.py`, 280 lines) follows an identical architecture to the EMA Agent but applies to Simple Moving Averages. Its crossover recognition includes the Golden Cross (SMA 50/200), recognised as the highest-reliability long-term trend signal for Gold. The agent de-weights rapid crossover combinations (e.g., SMA 10/20) due to their historically poor signal-to-noise ratio on daily Gold data.

4.4.5 Bollinger Bands Agent

The Bollinger Agent evaluates price position within the Bollinger Band envelope:

1. **Band Position Rules (40 points):** The agent normalises the close price's position within the bands to a 0–1 range (`position = (close - lower) / (upper - lower)`). For BUY: a position at or below the lower band earns 40 points; 0.15 proximity earns 32 points; up to 0.3 earns 20 points. SELL scoring follows a symmetric mapping for the upper band.

2. **Historical Win-Rate (40 points):** Band-touch events (close ≤ lower for BUY, close ≥ upper for SELL) are evaluated on the 5-bar, 0.5% forward horizon.

3. **Squeeze Detection (20 points):** The agent computes the `bb_bandwidth` as `(upper - lower) / middle` and compares the current bandwidth to its 50-bar rolling distribution. A bandwidth below the 20th percentile signals a Bollinger Squeeze — a volatility contraction that historically precedes explosive directional breakouts — earning a full 20 points.

4.5 Orchestrator Agent

The `OrchestratorAgent` class serves as the central coordinator of the AURUM pipeline. It is responsible for four critical operations:

4.5.1 Parallel Agent Dispatch

Parsed conditions are dispatched to their respective Specialist Agents using `ThreadPoolExecutor` with up to 5 workers (matching the number of agents). Since agents are stateless for evaluation purposes, concurrent execution is safe and reduces end-to-end latency by approximately 4× compared to sequential evaluation. The routing logic maps the condition's `indicator` field (uppercased) to a pre-instantiated agent lookup table.

```
    with ThreadPoolExecutor(max_workers=5) as executor:
        future_to_key = {
            executor.submit(self._evaluate_single, agent, condition,
df): key
            for key, agent, condition in work_items
        }
```

Failed agent evaluations are caught and replaced with zero-score sentinel `AgentResult` objects, ensuring partial pipeline failures do not crash the entire evaluation.

4.5.2 Weighted Score Computation

The Orchestrator maintains empirically derived weights reflecting each indicator's relative reliability on Gold:

Agent	Weight	Rationale
RSI	0.25	Gold is highly momentum-driven; RSI extremes are the most reliable reversal signals

Agent	Weight	Rationale
EMA	0.22	Dynamic trend following; EMA crossovers are widely trusted institutional signals
MACD	0.20	Strong momentum confirmation capability for Gold's trending regimes
BB	0.18	Volatility measurement provides unique information orthogonal to momentum
SMA	0.15	Longer-term trend context; slower-moving and less responsive to Gold's volatility

Table 2. AURUM agent weight distribution for Gold (XAUUSD).

The weighted score formula normalises by the sum of participating agents' weights, ensuring that strategies using a subset of indicators are not artificially penalised:

```
    Final Score = Σ(Agent_Score_i × Weight_i) / Σ(Weight_i)    for all
participating agents
```

4.5.3 Conflict Detection

The Orchestrator's `detect_conflicts()` method implements three pattern-matching checks:

1. **Action Alignment Mismatch (HIGH severity):** Any agents signalling BUY while others signal SELL (excluding NEUTRAL agents) generates a HIGH-severity conflict. This is the most critical contradiction — it means the strategy is internally self-defeating.
2. **RSI Overbought + EMA Uptrend (MEDIUM severity):** Triggered when the RSI Agent reports a value above 70 (exhaustion/overbought) while the EMA Agent aligns BUY (uptrend). This pattern signals elevated reversal risk despite a technically intact trend.
3. **MACD Bearish + RSI Oversold (MEDIUM severity):** Triggered when the MACD Agent aligns SELL (bearish momentum) while RSI is below 30 (oversold). This ambiguity — momentum collapse versus washed-out conditions — requires human judgement.

Conflict Penalties: HIGH-severity conflicts degrade the composite score by 15% per conflict (clamped to a minimum 50% retention). MEDIUM-severity conflicts apply a 5% penalty per conflict (clamped to 70% retention).

4.5.4 Suggestion Aggregation

Suggestions from all agents and conflict warnings are collected, deduplicated, and priority-ordered - conflict-derived suggestions appear first, followed by agent-level recommendations. This produces an actionable checklist for the trader.

4.6 Backtesting Engine

The `BacktestEngine` class performs vectorised strategy simulation. The engine operates under the following parameters:

Parameter	Value	Justification
Initial Capital	$10,000 USD	Standard retail account simulation baseline
Commission	0.1% per trade	Approximate Gold spread-adjusted commission
Forward Hold	5 bars	Captures the short-term directional edge of indicator signals

Table 3. Backtesting engine parameters.

4.6.1 Signal Mask Construction

Each parsed condition is translated into a boolean `pd.Series` mask via the `_condition_mask()` method, which dynamically computes any missing indicators using `pandas-ta`. All condition masks are combined with AND logic to produce a composite entry mask — a trade fires only when **every** condition is simultaneously satisfied.

The engine supports five indicator mask types:

- **RSI:** Direct threshold comparison against the RSI column.
- **EMA/SMA:** Price-vs-moving-average comparisons and period-crossover detection.
- **MACD:** Line-vs-signal comparisons using pre-computed or dynamically-generated MACD columns.
- **Bollinger Bands:** Close-vs-band boundary comparisons.

4.6.2 Trade Simulation

The `_simulate_trades()` method walks the signal mask sequentially, opening positions at each signal bar and closing them after the `FORWARD_HOLD` period. No overlapping trades

are permitted - the index advances past the hold period before scanning for the next signal. Per-trade return is computed as:

```
BUY return  = (exit_price - entry_price) / entry_price - commission
SELL return = (entry_price - exit_price) / entry_price - commission
```

4.6.3 Performance Metrics

The engine reports seven quantitative metrics from the trade list:

- **Total Return (%):** Portfolio gain/loss from initial capital.
- **Win Rate (%):** Fraction of profitable trades.
- **Total Trades:** Number of non-overlapping trades generated.
- **Sharpe Ratio:** Annualised mean-to-volatility ratio (`mean(returns) / std(returns) × √252`).
- **Maximum Drawdown (%):** Largest peak-to-trough decline in the equity curve.
- **Profit Factor:** Gross profit / gross loss (∞ when no losing trades).
- **Final Value:** Terminal portfolio balance.

An equity curve dictionary (`{date: balance}`) is produced for the Plotly line chart.

4.7 Dashboard Presentation Layer

The Streamlit dashboard (`dashboard/app.py`, 963 lines) assembles the full AURUM pipeline into a production-grade interactive UI.

4.7.1 Design System

The dashboard implements a custom dark-themed design system using CSS injection:

- **Background:** GitHub Dark palette (`#0D1117` primary, `#161B22` cards)
- **Accent:** Gold gradient (`#FFD700` → `#B8860B`)
- **Status Colours:** `#00FF88` (positive/green), `#FF4444` (negative/red), `#58A6FF` (informational blue)
- **Typography:** JetBrains Mono for the strategy input area, system fonts elsewhere

Custom CSS classes style the score ring (circular border with centred score text), conflict boxes (colour-coded left borders by severity), and button hover effects (translateY transform with box-shadow glow).

4.7.2 Lazy Import Strategy

To minimise cold-start time on resource-constrained environments (notably Replit free tier), all heavy imports (`yfinance`, `plotly`, `pandas`, agent classes, backtest engine) are deferred to lazy-loading wrapper functions that are invoked only when their functionality is required.

4.7.3 Data Caching

Streamlit's `@st.cache_data` decorator is applied with appropriate TTL values:

- Gold spot price: 15-minute cache (TTL=900s)
- Historical OHLCV data: 1-hour cache (TTL=3600s)
- Indicator computation: 1-hour cache

4.7.4 Layout Structure

The dashboard is organised into:

1. **Header Banner:** Gradient-styled title with live TradingView widget embed (`OANDA:XAUUSD`).
2. **Sidebar:** Health check panel showing API connection status (yfinance, Alpha Vantage, Anthropic), latest Gold price, and environment metadata.
3. **Left Panel:** Strategy text area, backtest period slider, evaluation trigger button. Post-evaluation: score ring, agent metric cards (3-column grid), and conflict warning boxes.
4. **Right Panel:** Three-tab layout:
    - **Price Chart:** 90-day candlestick chart with EMA overlays.
    - **Backtest Results:** 4-column metric cards row + equity curve.
    - **Agent Reports:** Expandable per-agent panels showing feedback, suggestions, win rate, and action alignment.

---

5. Results and Discussion

5.1 Functional Validation

The AURUM system was validated against a corpus of test scenarios spanning standard, edge-case, and adversarial strategy configurations.

5.1.1 Standard Strategy Evaluation

Test Case 1: Golden Cross (SMA 50/200):
The canonical "Buy when SMA50 crosses above SMA200" strategy was correctly parsed into a single SMA crossover condition. The SMA Agent recognised the (50, 200) combination as the highest-reliability crossover pattern and awarded correspondingly high rule-based credit. The composite score fell within the GOOD to EXCELLENT band (70–85), consistent with the empirical track record of the Golden Cross on Gold over the 2019–2024 period.

Test Case 2: Multi-Indicator Confluence:
The strategy "Buy Gold when RSI is below 35 and EMA20 crosses above EMA50 and MACD bullish crossover" was correctly decomposed into three conditions routed to three specialist agents. All three agents aligned BUY, yielding no conflicts. The Orchestrator's weighted aggregation produced a composite score reflecting the confluence of momentum (RSI), trend (EMA), and momentum-confirmation (MACD) signals.

5.1.2 Contrarian Strategy Penalisation

Test Case 3: Buying Overbought:
The adversarial strategy "Buy when RSI > 80" was correctly penalised by the RSI Agent. The rule-based scoring component awarded 0 points (RSI > 80 in BUY context maps to the "Not Oversold" zone). Historical win-rate analysis confirmed that buying at RSI > 80 on Gold yields sub-50% success rates, resulting in a WEAK or POOR final rating with targeted improvement suggestions ("Try RSI < 30 for classic oversold").

5.1.3 Conflict Detection

Test Case 4: RSI Oversold + MACD Bearish:
When tested with "Buy when RSI below 30 and MACD bearish crossover", the parser correctly generated two conditions RSI (BUY-direction) and MACD (SELL-direction due to bearish crossover). The Orchestrator detected an action-alignment mismatch (HIGH severity) between the RSI Agent (BUY alignment due to oversold conditions) and the MACD Agent (SELL alignment due to bearish momentum), issued a descriptive warning explaining the contradiction, and applied a 15% score penalty.

5.2 Performance Characteristics

Metric	Measured Value
Strategy parse time (regex mode)	< 5 ms
Strategy parse time (Claude fallback)	1.2–3.0 s
Agent evaluation (5 agents, parallel)	0.8–2.0 s
Backtest execution (5 years daily data)	0.3–0.8 s
Total pipeline (regex mode)	2–4 s
Dashboard cold start (Replit)	4–6 s

Table 4. End-to-end performance benchmarks.

The regex-first parsing strategy ensures that the majority of standard strategy inputs are processed with sub-5ms latency, with the Claude fallback adding 1–3 seconds only for genuinely complex natural language constructions. The parallel agent dispatch via `ThreadPoolExecutor` reduces agent evaluation time from ~5 seconds (sequential) to under 2 seconds.

5.3 Backtesting Validity

The backtesting engine was validated by constructing known-outcome scenarios where the composite signal mask contains a predetermined number of entries with calculable returns. The engine's reported Win Rate, Total Return, and Profit Factor were verified against manual computation across multiple test configurations, confirming arithmetical correctness of the simulation loop and commission deduction logic.

One limitation of the current backtest design is the fixed 5-bar forward hold period, which does not adapt to the volatility regime or the specific indicator being traded. This limitation is acknowledged and addressed in the Future Work section.

5.4 Limitations

1. **Asset Restriction:** The current implementation exclusively evaluates Gold (XAUUSD). Indicator weights and domain rules are empirically calibrated for Gold's market dynamics and would require re-derivation for other assets.

2. **Fixed Hold Period:** The backtester's 5-bar forward hold does not accommodate indicator-specific holding strategies (e.g., trailing stops, dynamic exits).

3. **Offline ML Models:** The win-rate estimation is computed as a lookup-style historical scan rather than a trained ML classifier, limiting generalisation beyond the observed data distribution.

4. **No Position Sizing:** The engine assumes full capital deployment per trade without Kelly criterion-based or volatility-adjusted position sizing.

---

6. Conclusion and Future Work

6.1 Conclusion

This paper presented AURUM, a multi-agent artificial intelligence system that bridges the gap between raw backtesting mathematics and explainable trading strategy evaluation for the Gold (XAUUSD) market. By decomposing strategy analysis into five autonomous Specialist Agents, each independently scoring conditions through a uniform three-component model of rules, historical validation, and divergence detection, the system provides granular, per-indicator feedback that empowers traders to understand not merely *whether* a strategy works, but *why* it succeeds or fails at each indicator level.

The concurrent orchestration architecture, dual-mode NLP parser, multi-pattern conflict detection subsystem, and vectorised backtesting engine collectively deliver end-to-end strategy evaluation within 2–4 seconds, enabling rapid iterative refinement. The interactive Streamlit dashboard presents this analysis through an integrated interface combining live price data, agent scorecards, conflict warnings, quantitative backtest metrics, and actionable improvement suggestions.

Empirical validation confirmed that AURUM correctly classifies canonical high-probability setups, appropriately penalises contrarian configurations, and successfully intercepts inter-agent contradictions that would go undetected in conventional backtesting platforms.

6.2 Future Work

1. **Trained ML Classifiers per Agent:** Replace the current lookup-style win-rate scan with pre-trained Logistic Regression or Gradient Boosting models (`scikit-learn`) trained on engineered feature sets (indicator value, slope, momentum, volume ratio) per agent, enabling generalisation to out-of-sample periods.

2. **Multi-Asset Expansion:** Extend support beyond Gold to indices (S&P 500, NASDAQ-100), forex pairs (EUR/USD), and cryptocurrency markets, with dynamically re-derived agent weights per asset class.

3. **Automated Parameter Optimisation:** Integrate Optuna-powered hyperparameter search to allow the Orchestrator to automatically optimise user-specified thresholds (e.g., shifting RSI 30 to RSI 28) to maximise the Sharpe Ratio or Profit Factor.

4. **Dynamic Exit Strategies:** Replace the fixed 5-bar hold with ATR-based trailing stops, EMA-based exits, and indicator-reversal exits that adapt to market volatility.

5. **Risk-Adjusted Position Sizing:** Implement Kelly criterion and volatility-normalised position sizing to produce more realistic equity curves and risk metrics.

6. **Strategy Comparison Mode:** Enable side-by-side evaluation of two user strategies with differential highlighting of scoring, conflict, and backtest divergences.

7. Technology Stack Summary

Layer	Technology	Version	Purpose
Language	Python	≥ 3.10	Core runtime
Data (Historical)	yfinance	≥ 0.2.40	Multi-year daily OHLCV for XAUUSD
Data (Real-time)	Alpha Vantage API	—	Live XAU/USD exchange rate
Data (Cache)	SQLite	built-in	Persistent OHLCV and indicator cache
Indicators	pandas-ta	≥ 0.3.14b0	RSI, EMA, SMA, MACD, Bollinger Bands
Validation	Pydantic	≥ 2.6.0	Condition and result schema enforcement
NLP Fallback	Anthropic Claude API	claude-sonnet-4-20250514	Complex natural language parsing
Dashboard	Streamlit	≥ 1.32.0	Interactive web-based UI
Charting	Plotly	≥ 5.19.0	Candlestick, equity curve rendering
Backtesting	NumPy, pandas	< 2.0.0, 1.5.3	Vectorised strategy simulation
Machine Learning	scikit-learn	≥ 1.4.0	Future classifier training pipeline
Scheduling	APScheduler	≥ 3.10.0	Periodic price refresh
Testing	pytest	≥ 8.0.0	Unit and integration test framework

Table 5. Complete technology stack of the AURUM system.

8. Source Code Repository

The complete source code, documentation, and deployment configuration files for AURUM are publicly available at:

Directory Structure:

```
    AURUM/
    ├── agents/
    │   ├── base_agent.py         # Abstract base class +
AgentMessage/AgentResult
    │   ├── rsi_agent.py          # RSI specialist (368 lines)
    │   ├── ema_agent.py          # EMA specialist (283 lines)
    │   ├── sma_agent.py          # SMA specialist (280 lines)
    │   ├── macd_agent.py         # MACD specialist (326 lines)
    │   ├── bollinger_agent.py    # Bollinger Bands specialist (305
lines)
    │   └── orchestrator.py       # Master orchestrator (472 lines)
    ├── parser/
    │   └── strategy_parser.py    # Dual-mode NLP parser (477 lines)
    ├── backtest/
    │   └── engine.py             # Vectorised backtesting engine (325
lines)
    ├── data/
    │   ├── fetcher.py            # yfinance + Alpha Vantage API (138
lines)
    │   └── storage.py            # SQLite PriceStorage (274 lines)
    ├── indicators/
    │   ├── calculator.py         # pandas-ta indicator wrapper (77
lines)
    │   └── ta_compat.py          # Version compatibility shim (18
lines)
    ├── dashboard/
    │   └── app.py                # Streamlit dashboard (963 lines)
    ├── requirements.txt
    └── README.md
```

References

[1] World Gold Council, "Gold Demand Trends Full Year 2024," World Gold Council Report, 2024.

[2] C. D. Kirkpatrick and J. R. Dahlquist, *Technical Analysis: The Complete Resource for Financial Market Technicians*, 3rd ed. Upper Saddle River, NJ, USA: FT Press, 2016.

[3] J. W. Wilder, *New Concepts in Technical Trading Systems*. Greensboro, NC, USA: Trend Research, 1978.

[4] M. Wooldridge, *An Introduction to Multi-Agent Systems*, 2nd ed. Chichester, UK: John Wiley & Sons, 2009.

[5] B. LeBaron, "Agent-based computational finance," in *Handbook of Computational Economics*, vol. 2, L. Tesfatsion and K. L. Judd, Eds. Amsterdam, The Netherlands: Elsevier, 2006, pp. 1187–1233.

[6] R. Bianchi, M. Drew, and J. Fan, "Combining momentum with reversal in commodity futures," *Journal of Banking & Finance*, vol. 59, pp. 423–444, 2015.

[7] G. Appel, *Technical Analysis: Power Tools for Active Investors*. Upper Saddle River, NJ, USA: FT Press, 2005.

[8] O. Polakow, "vectorbt: Fast, vectorized backtesting in Python," GitHub Repository, 2020. [Online]. Available: https://github.com/polakowo/vectorbt